

PROBLEMA:	Introdução ao Diagrama de Classes	DATA:	14/08/2026
PILOTO (A):	Andrei Luis	COPILOTO (A):	David Cairo
QA:	Gabriel Daniel Silva	ARQUITETO (A):	Mateus Oliveira
SCRUM MASTER:	Victor Ferreira		

Objetivo: Dominar a aplicação prática de membros estáticos (static) implementando a camada de acesso a dados (Repositórios) com o Padrão de Projeto *Singleton*.

Fatos:

(O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 1 e o Arquiteto documenta no repositório)

1. **A Restrição Arquitetural:** Segundo a introdução da OAT e a seção "Motor de Armazenamento", quais são as tecnologias e recursos do Spring Boot (relacionados a banco de dados e injeção de dependência) que são estritamente proibidos nesta fase inicial?
 - **Fato Extraído:**
O uso de injeção de dependência do Spring Boot é estritamente proibido.
2. **O Padrão de Armazenamento:** A OAT exige a criação de repositórios manuais (ex: [PecaRepository](#)). Qual Padrão de Projeto deve ser obrigatoriamente utilizado para implementá-los e qual método deve conceder acesso exclusivo a eles?
 - **Fato Extraído:** O padrão de projeto obrigatório é o Singleton. O método que concede acesso exclusivo à instância é o getInstance()

Brainstorm Investigativo:

(Questões e Ideias guiadas pelo Scrum Master)

- **O Construtor Privado:** Por que o construtor dessas classes repositório precisa ser *private*? O que isso impede que os programadores façam no futuro?

O construtor é *private* pra impedir que qualquer outra classe do sistema crie uma nova instância do repositório.

- **A Sobrevivência dos Dados:** Como as listas internas de persistência vão sobreviver e se manter as mesmas enquanto o servidor estiver rodando? Qual é o papel do atributo *static* nesse comportamento?

Para manter os dados íntegros enquanto a aplicação roda, os repositórios devem ser implementados usando o Padrão de Projeto Singleton, acessados exclusivamente pelo método `getInstance()`. O papel do atributo *static* é fundamental para garantir que esses dados não sejam apagados quando novos objetos forem criados.

Cenários de Teste:

(Liderado pelo QA)

- Testar se o programa falha em tempo de compilação ao tentarmos instanciar o `PecaRepository` usando a palavra-chave `new` fora da própria classe.

Entrega do Encontro:

- **Código:** As classes `PecaRepository.java` e `ServicoRepository.java` implementando o padrão Singleton de forma manual, contendo as listas internas.
- **Diagrama:** Atualizar o Diagrama de Classes, evidenciando o sublinhado em membros UML que indica que os atributos e métodos `getInstance()` são estáticos.

PROBLEMA:	Introdução ao Diagrama de Classes	DATA:	28/08/2026
PILOTO (A):	Victor Barbosa Da Hora	COPILOTO (A):	Andrei Luís Santos de Souza
QA:	David Cairo	ARQUITETO (A):	Gabriel Daniel
SCRUM MASTER:	Mateus Oliveira Sousa		

Objetivo: Mapear rotas na Web API estilo REST, compreender as anotações do framework Spring Boot e estabelecer os relacionamentos lógicos entre a interface web e os repositórios.

Fatos:

(O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 1 e o Arquiteto documenta no repositório)

1. **Rotas e Controladores:** De acordo com a seção "Roteamento REST e Controladores" da OAT, qual é o padrão estrutural que as rotas da nossa aplicação devem seguir para expor os dados das peças e serviços?
 - **Fato Extraído:** O padrão estrutural exigido é o RESTful, expondo os recursos em rotas no plural como /api/pecas e /api/servicos.
2. **A Semântica de Sucesso:** A OAT determina que a API não pode simplesmente devolver os dados puros. Quais são os Status HTTP exatos que devem ser retornados para Criação com sucesso, Listagens/Atualizações, e Deleções bem sucedidas?
 - **Fato Extraído:** Criação com sucesso: 201 Created
 Listagens e Atualizações com sucesso: 200 OK
 Deleções bem-sucedidas: 204 No Content

3. **A Semântica de Erro:** Qual Status HTTP a OAT exige que seja codificado e retornado caso uma operação de busca ou deleção não encontre o ID solicitado pelo cliente?
- **Fato Extraído:** O status exigido é o 404 Not Found.

Brainstorm Investigativo:

(Questões e Ideias guiadas pelo Scrum Master)

- **Integração:** Como faremos com que o Controller peça para o Repository salvar o dado, lembrando que a OAT proíbe a injeção de dependência padrão do Spring?
Como o uso de `@Autowired` está proibido, a integração é feita obtendo a instância única do repositório diretamente via método estático do Singleton na declaração do atributo do Controller:
`private final PecaRepository pecaRepository = PecaRepository.getInstance();`
- **Relacionamentos no Diagrama:** Como representaremos no Diagrama de Classes a relação visual entre o `PecaController` e o `PecaRepository`?
A relação é representada por uma Associação Unidirecional (uma seta simples que sai do `PecaController` apontando para o `PecaRepository`). No `PecaRepository`, destacam-se o construtor privado (`- PecaRepository()`), a variável estática privada (`- instance: PecaRepository`) e o método estático público (`+ getInstance(): PecaRepository`).

Cenários de Teste:

(Liderado pelo QA)

- Validar a execução de uma requisição **POST** que deve resultar na gravação em memória e em um retorno **201 Created**.

Execução: Enviar uma requisição POST para `http://localhost:8080/api/pecas` contendo no corpo (JSON) os dados da peça (código de barras, quantidade, valores, categoria).

Resultado Esperado: A API grava a peça na lista em memória da instância Singleton e responde com o Status HTTP 201 Created, devolvendo o JSON com o id gerado e as datas preenchidas.

- Validar o retorno manual de **404 Not Found** ao tentar buscar uma Peça cujo ID ainda não foi cadastrado no Singleton.

Execução: Enviar uma requisição GET para `http://localhost:8080/api/pecas/999` (ID inexistente na memória).

Resultado Esperado: A API identifica que a busca no repositório retornou vazia e devolve o Status HTTP 404 Not Found.

Entrega do Encontro:

- **Código:** `PecaController.java` e `ServicoController.java` com endpoints implementados para suprir as 8 User Stories.
- **Artefatos:** *Collection* (Postman ou Insomnia) exportada com as rotas criadas. O Arquiteto finaliza o Diagrama de Classes.